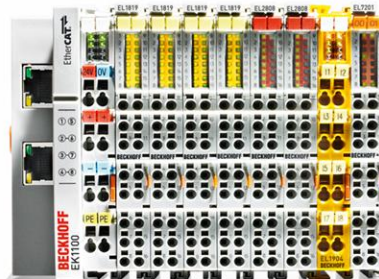


PLC_MOTION_LAYER

TwinCAT project

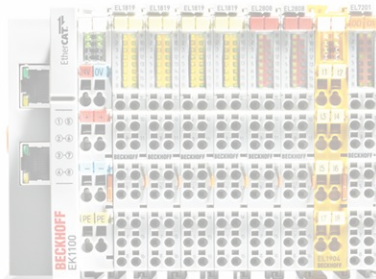
BECKHOFF



Daniel Hauer

Introduction to motion layer approach

- motivation for a motion layer
- use cases
- specific construction



Motivation:

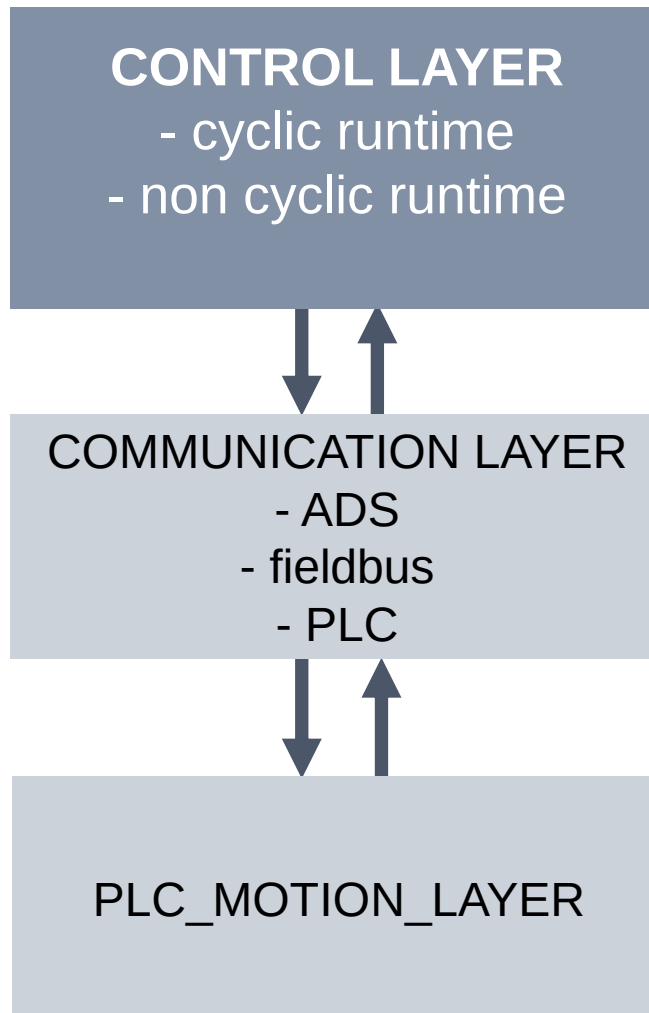
- **Centralized control:**
 - Sync communication
 - Async communication
 - Agnostic to communication technology
- **Decentralized execution of motion tasks**
 - Data consistency in cyclic multi task environments
 - Local machine with modular design specifications
 - Unified procedure for modular software architecture
 - Use of top layer consistent through different architectures

Motivation:

- **Use of TwinCAT supported/updated libraries**
 - Tc2_MC2
 - Tc2_MC2_Drive
 - Tc2_NC
 - Tc2_NCI
 - Tc2_PlcInterpolation
- **Open code base**
 - Migration to Tc3 MC in preparation
 - Customer/user specific changes possible
 - Access to code
 - Conversion to library possible by customer/user
- **Compiled PLC**
 - Source code is not on shipped machine

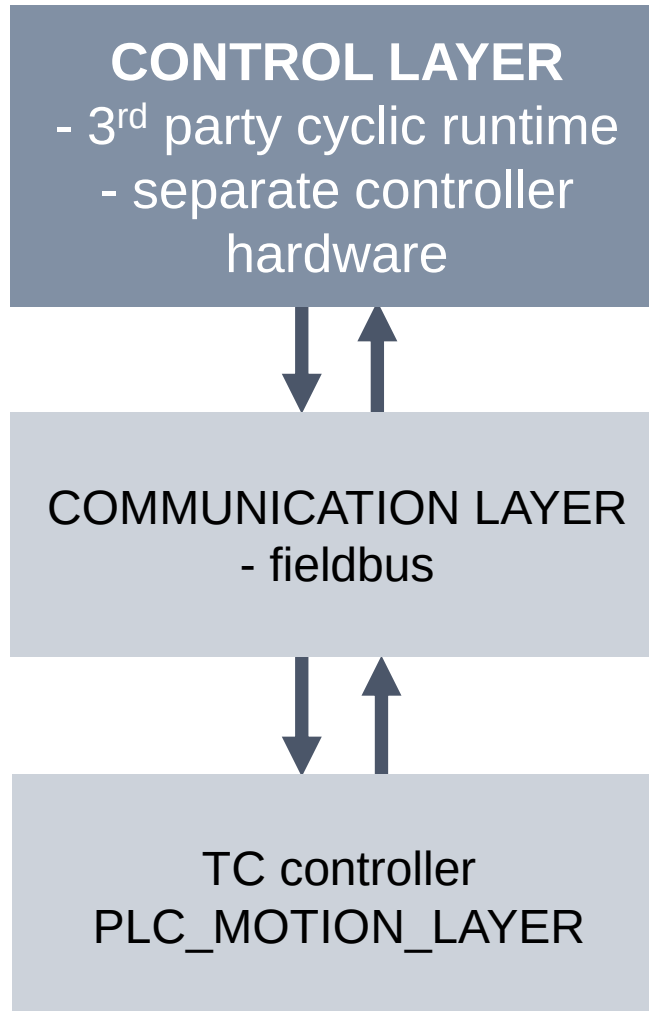
Motivation:

- Use of TwinCAT motion without detailed coding knowledge
- Transparency of communication layer
- Code base shall remain independent of control layer
- Configurable Options for specific libraries / TC functions
- Balanced load for configurable options in machine layout
- Stable cpu use for XFC applications



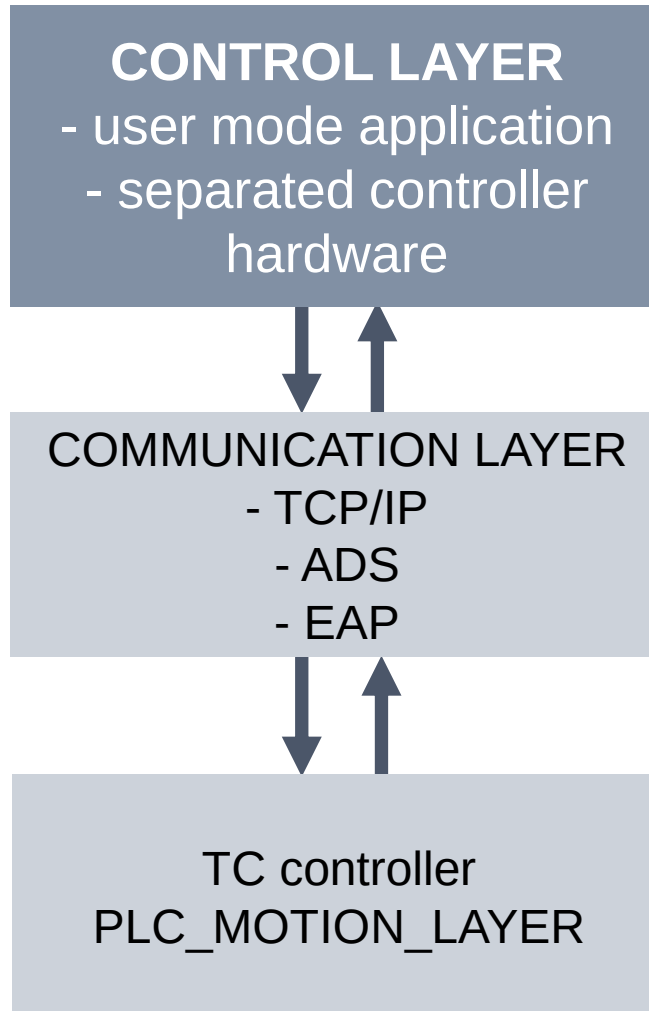
Use cases:

- Separate controller for machine logic
- Any fieldbus (EtherCAT, Profi...)
- Connected through TwinCAT mappings
- Execution of motion tasks in PLC_MOTION_LAYER
TwinCAT controller



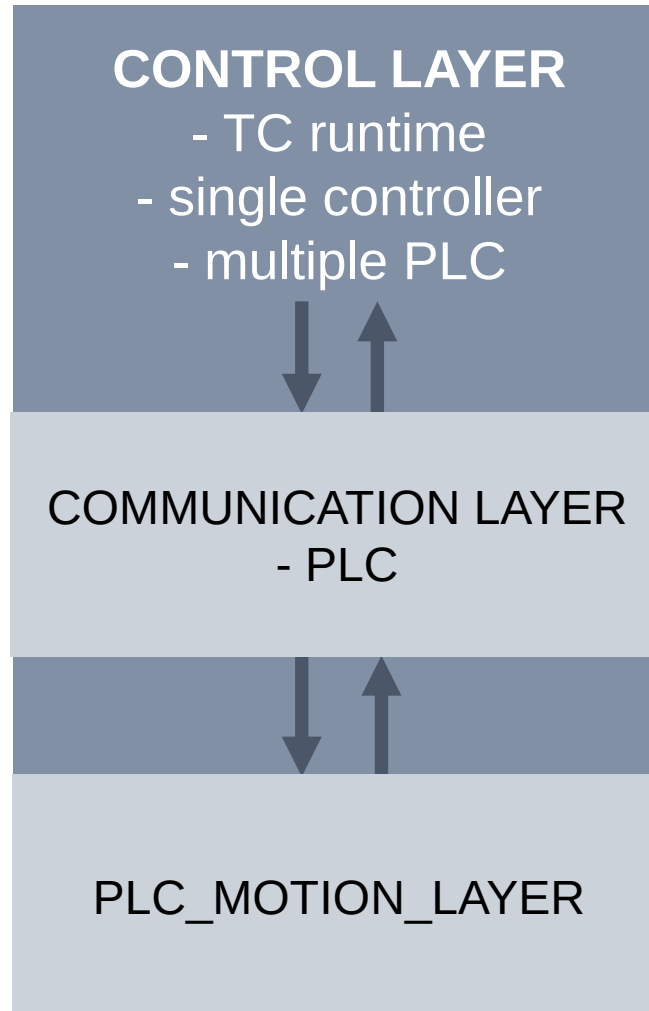
Use cases:

- Separate controller for machine logic
- Any network
- Connected through TwinCAT mappings
- Execution of motion tasks in PLC_MOTION_LAYER TwinCAT controller



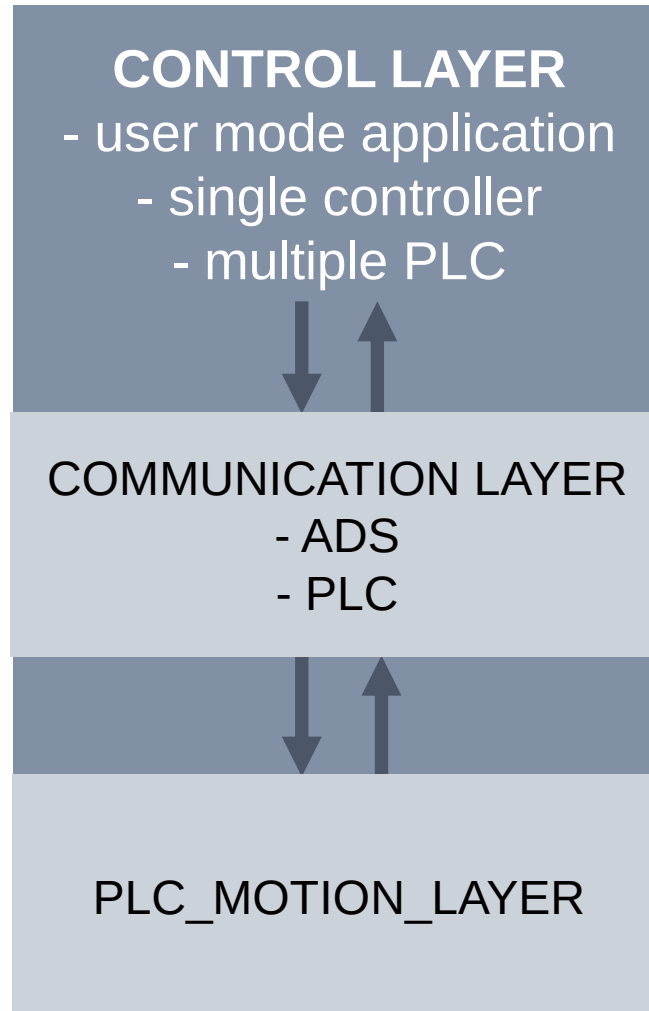
Use cases:

- One controller for machine logic
- Multiple PLC for machine logic
- Connected through TwinCAT mappings
- Execution of motion tasks



Use cases:

- One controller for machine logic
- User mode application AND/OR multiple PLC
- ADS for symbol access by user mode application
- TwinCAT mapping for connecting multiple PLCs for specific application purposes
- Execution of motion tasks



Specific construction:

- TwinCAT PLC project
- Use of specific syntactic code behaviour
- Software design
- Compiler defines / pragmas
- Logging system

TwinCAT project:

- Default TwinCAT project
 - Adjust core settings to target hardware
 - Add NC/PtP
 - Optional add NCI channel
- TwinCAT PLC
 - Add existing Item: PLC_MOTION_LAYER
 - Add task reference
 - Option: add compiler defines (NCI, CAM, BSD)
- Adjust constants in:
 - PLC_MOTION_LAYER/PLC_CONSTANT
- Compile
 - PLC_MOTION_LAYER Instance mapping is built

specific syntactic code behaviour:

- C like state machines
 - State changes **need not** consume one PLC cycle
- Same cycle response to command on cyclic interface
 - → since we're not using C (unfortunately, but ST in TwinCAT3 is almost as beautiful)
 - → cases have to be 'broken up'
 - → take a close look, it probably looks weird, but it is fast AND safe to use in combination with E_PROGRESS
 - → look at FB_MaAxisBase.MovePosBuffer try to do it with IF conditionals, then come back and take a second look ;-)

specific syntactic code behaviour:

- OnChange detection for new commands
 - Cyclic check whether the command has changed
- State always carries offset about progress of command (E_PROGRESS) (example follows in a few pages)
- Library FBs are called within states
 - FBs are called when required and not a cycle longer (same is true for all Ctrl-Wrappers)
 - Working with empty cyclic calls is the best way to build voodoo software, just don't do it!

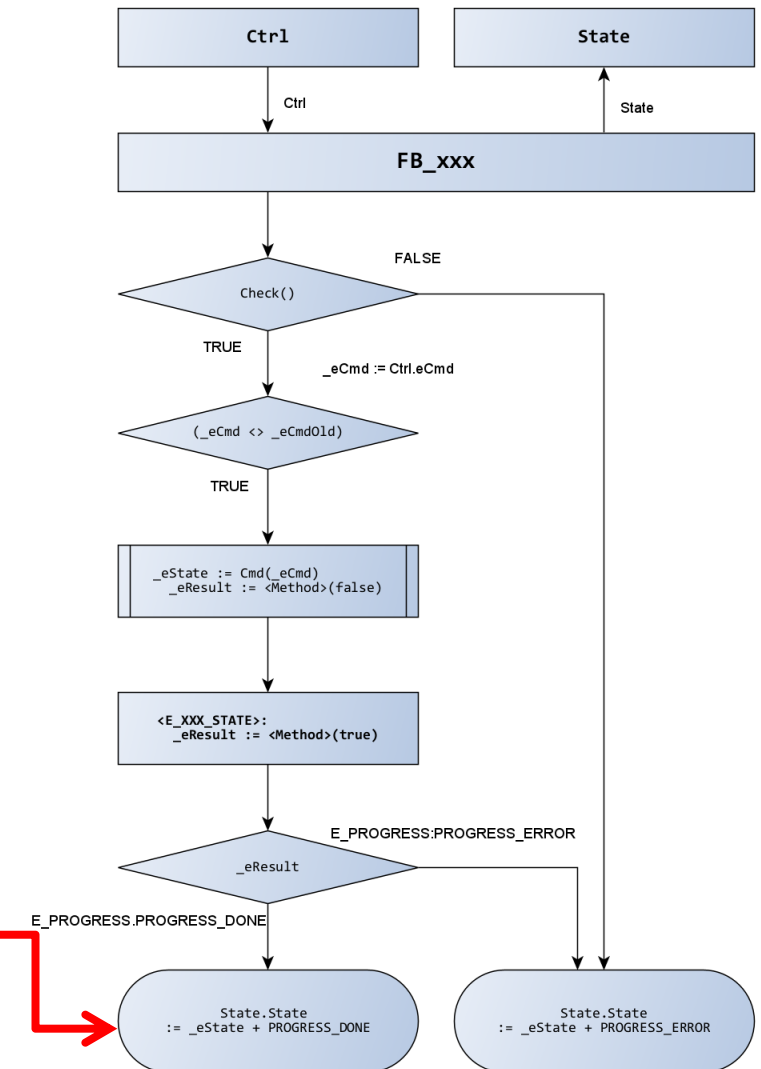
```
_eCmd := _Ctrl^[_nAxisIndex].eCmd;  
//-----  
//-----  
// OnChange react to command  
//-----  
IF (_eCmd <> _eCmdOld)  
THEN  
    // get busy  
    _eState := Cmd(_eCmd); // get execution state for command  
    _eCmdOld := _eCmd;  
    LogControl();  
END_IF
```

Software Design:

- Every TwinCAT function has dedicated wrapper
 - Separate namespaces
 - Optional library binding
- Cross communication via interfaces
 - NCI, CAM, XFC use interfaces in order to enable optional binding
 - If compiler define is not set, empty interfaces are used instead of instances
- Ctrl/State structures for commanding required function
 - PtP – ctrl/state
 - NCI – ctrl/state
 - CAMMING – ctrl/state
- Parameter structures carry required data for commanded function
 - PtP – (SetPos, SetVelo, SetAcc, MasterAxisIndex...)
 - NCI – (AxisGroupId, AxisIndex, MFunc, RParameter...)
 - CAMMING – (MasterAxisIndex, TableId...)

Software Design:

- State / Ctrl structures
 - Establishes unified access
 - Commands can simply be 'dropped' into Ctrl datafield
 - Enables asynchronous communication with PLC_MOTION_LAYER (e.g. C# via ADS, C/C++ via ADS, ADS over MQTT, ...)
 - Enables cyclic communication with PLC_MOTION_LAYER since structures can easily be mapped onto any cyclic fieldbus TwinCAT supports (EtherCAT, Profinet, CanOpen, EAP, ...)
 - State is updated by PLC_MOTION_LAYER, so you can move on doing other stuff, come back and check completion/error.
- State feedback for cyclic class wrappers
 - **Always** combined with E_PROGRESS
 - You can filter your response by a simple modulo division
 - The result may be your entry point for your reaction to State.



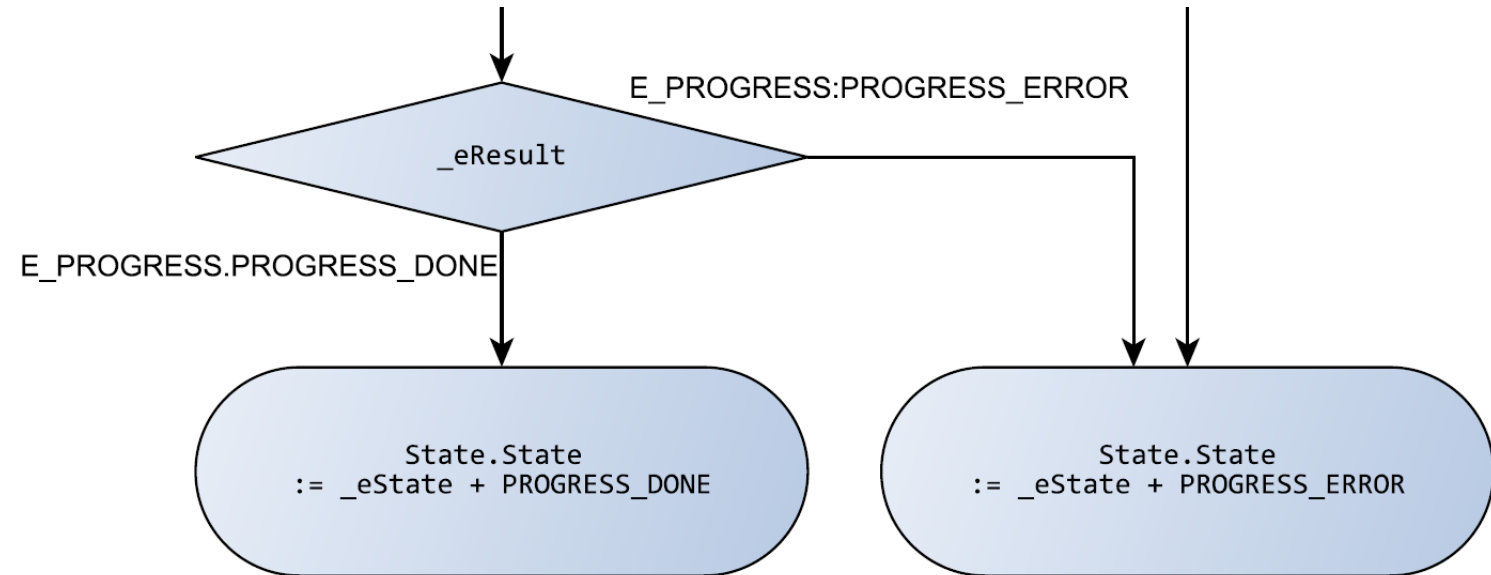
Software Design:

- E_PROGRESS
 - How far along is the command you just 'dropped'?
 - This enum shall help you to build an answer state machine and not just a simple **IF** conditional
 - This enum shall be used everywhere and I recommend you use it in your external control layer

```
E_PROGRESS  ▢ ×
1  {attribute 'qualified_only'}
2  //{attribute 'strict'}
3  {attribute 'to_string'}
4  TYPE E_PROGRESS :
5  (
6    // progress has 2 use cases
7    // 1. as offset to cyclic interface's state
8    //    for the requested command/function
9    //    e.g. State := <Enum equivalent to eCmd> + E_PROGRESS
10   //
11   // 2. as state feedback (result) from (any) method
12   PROGRESS_INVALID,
13   PROGRESS_NOT_EXIST      := 100,
14   PROGRESS_INIT           := 1000,
15   PROGRESS_BUSY           := 2000,
16   PROGRESS_PREPARE        := 3000,
17   PROGRESS_STARTUP        := 4000,
18   PROGRESS_CHECK          := 5000,
19   PROGRESS_OCCUPIED       := 6000,
20   PROGRESS_WORKING        := 7000,
21   PROGRESS_STILL_WORKING  := 8000,
22   PROGRESS_ERROR          := 9000,
23   PROGRESS_DONE           := 10000
24 )UINT;
25 END_TYPE
```


Software Design:

- E_PROGRESS (example)
 - How far along is the command you just 'dropped'?
 - This enum shall help you to build an answer state machine
 - This enum shall be used everywhere and I recommend you use it in your extern control layer too.



Software Design:

- E_PROGRESS (example)
 - How far along is the command you just 'dropped'?
 - See example skeleton →
 - This enum shall help you to build an answer state machine
- This enum shall be used everywhere and I recommend you use it in your extern control layer too.

```
//-----  
//-----  
// react to operation modes (how to do it in an easy self documenting way)  
//-----  
//-----  
CASE (GVL_AXIS.State[i].eState MOD E_PROGRESS.PROGRESS_DONE)  
OF  
    E_AXIS_STATE.AXIS_INIT:  
        ;// do something after success  
  
    E_AXIS_STATE.AXIS_MOVE_POS::  
        ;// do something after success  
ELSE  
    CASE (GVL_AXIS.State[i].eState MOD E_PROGRESS.PROGRESS_ERROR)  
    OF  
        E_AXIS_STATE.AXIS_INIT:  
            ;// do something after success  
  
        E_AXIS_STATE.AXIS_MOVE_POS::  
            ;// do something after success  
    ELSE  
        ;// down the rabbit hole you go from here  
    END_CASE  
END_CASE
```

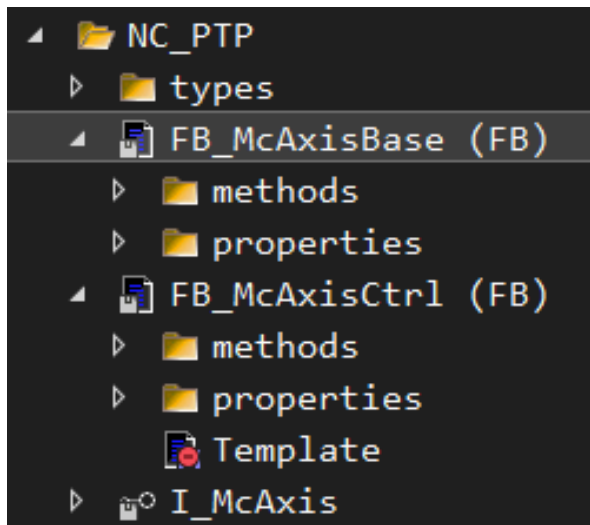
Software Design:

- E_PROGRESS (example log)
 - Different names
 - Same principle

eMessageInfo	2025-10-24-13:00:18.408	General_ID_0	General	30	ExampleEvalMachine	:TRANSPORT_GROUP_CLEAR:	PROGRESS_INIT
eMessageInfo	2025-10-24-13:00:18.418	General_ID_0	General	30	ExampleEvalMachine	:TRANSPORT_GROUP_CLEAR:	PROGRESS_BUSY
eMessageInfo	2025-10-24-13:00:18.458	General_ID_0	General	30	ExampleEvalMachine	:TRANSPORT_GROUP_CLEAR:	PROGRESS_PREPARE
eMessageInfo	2025-10-24-13:00:18.658	General_ID_0	General	30	ExampleEvalMachine	:TRANSPORT_GROUP_CLEAR:	PROGRESS_STARTUP
eMessageInfo	2025-10-24-13:00:18.858	General_ID_0	General	30	ExampleEvalMachine	:TRANSPORT_GROUP_CLEAR:	PROGRESS_CHECK
eMessageInfo	2025-10-24-13:00:18.888	General_ID_0	General	30	ExampleEvalMachine	:TRANSPORT_GROUP_CLEAR:	PROGRESS_OCCUPIED
eMessageInfo	2025-10-24-13:00:18.908	General_ID_0	General	30	ExampleEvalMachine	:TRANSPORT_GROUP_CLEAR:	PROGRESS_DONE

Software Design:

- Class building example:
 - **Library** wrapper as **ABSTRACT** BaseClass with Interface
- Ctrl/State wrapper extends BaseClass
 - NC_PTP example

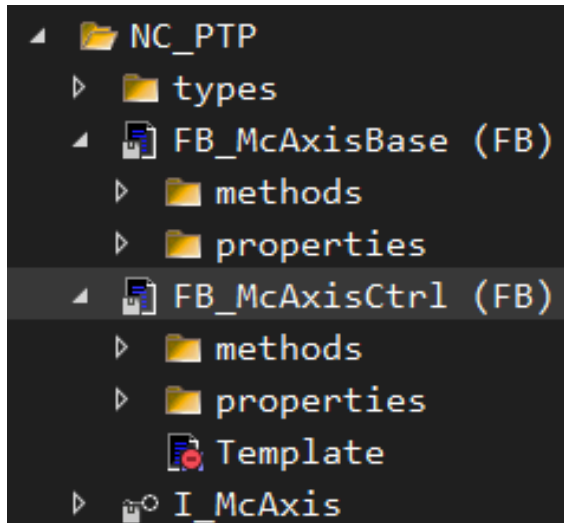


```

FB_McAxisBase
19
20 // last target position datafield
21 _rLastTargetPosition      : LREAL;
22
23 // name and encoder type are read by ADSREAD instance from NC
24 _sAxisName                 : STRING(30) := 'NO NAME'; // AxisName is limited to 30 chars,
25 _sEncoderType              : STRING(30) := 'NO TYPE';
26 _nEncoderType              : UDINT;
27 _eEncoderType              : E_AxisEncoderType;
28
29 // Motion Control FB Instances
30 _fbPower                   : Tc2_MC2.MC_Power;
31 _fbReset                   : Tc2_MC2.MC_Reset;
32 _fbHalt                    : Tc2_MC2.MC_Halt;
33 _fbStop                    : Tc2_MC2.MC_Stop;
34 _fbHome                    : Tc2_MC2.MC_Home;
35 _fbJog                     : Tc2_MC2.MC_Jog;
36 _fbMoveToPosition          : Tc2_MC2.MC_MoveAbsolute;
37 _fbMoveVelo                : Tc2_MC2.MC_MoveVelocity;
38
39 // used for buffered moves
40 _fbMoveVelocity            : ARRAY[1..2] OF Tc2_MC2.MC_MoveVelocity;
41 _fbMoveAbsolute            : ARRAY[1..2] OF Tc2_MC2.MC_MoveAbsolute;
42
43 // ADS read instance for reading startup information from NC
    
```

Software Design:

- Class building example:
 - Library wrapper as ABSTRACT BaseClass with Interface
 - Ctrl/State wrapper **extends** BaseClass
 - NC_PTP example

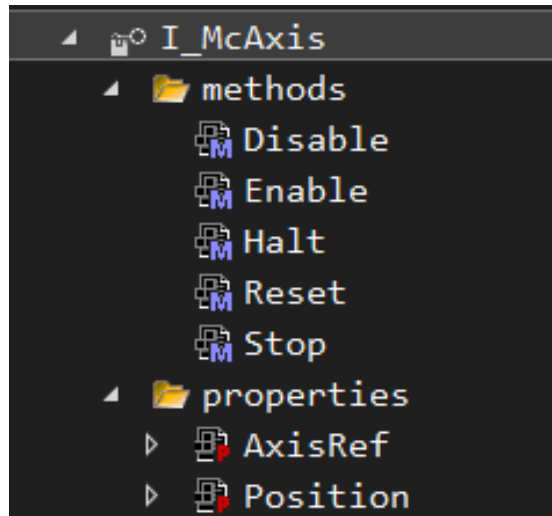


```

FB_McAxisCtrl  FB_McAxisBase
1  FUNCTION_BLOCK FB_McAxisCtrl EXTENDS FB_McAxisBase
2  VAR
3      _eCheck          : E_CHECK_STATE;  // pointer check
4
5      _eAxisCycle,
6      _eInit           : E_PROGRESS;      // cyclic AXIS_REF() and MC_Power check
7                                     // axis startup and init, get encoder type,
8
9      _nAxisIndex      : UINT;            // know thyself
10     _rTargetWindow    : LREAL := 0.1;    // TODO: get that from NC
11
12     // pointers to all; access required for coupling; access by index
13     _Axes              : REFERENCE TO ARRAY[1..PLC_CONSTANT.MAX_AXIS] OF AXIS_REF;
14
15     _Ctrl              : POINTER TO ARRAY[1..PLC_CONSTANT.MAX_AXIS] OF ST_AXIS_CTRL;
16     _State             : POINTER TO ARRAY[1..PLC_CONSTANT.MAX_AXIS] OF ST_AXIS_STATE;
    
```

Software Design:

- Class building example:
 - Library wrapper as ABSTRACT BaseClass **with** Interface
 - **Interface** for cross-communication between classes
 - NC_PTP example



```

FB_McAxisCtrl  FB_McAxisBase  X
1  FUNCTION_BLOCK ABSTRACT FB_McAxisBase IMPLEMENTS I_McAxis
2  VAR
3      _AxisRef          : REFERENCE TO Tc2_MC2.AXIS_REF;
4      _pCalibrationCam  : POINTER TO BOOL;
5
6      _nErrorId         : UDINT; // datafield for MC error
7
8      _MessageLevel     : E_MessageType;
9
10     // MC_Power inputs for feed fwd, feed bwd, override
11     _bEnablePositive   : BOOL := TRUE;
12     _bEnableNegative   : BOOL := TRUE;
13     _rAxisOverride     : LREAL := 100.0;
14
15     _rValueParameterRead : LREAL;
16     _fbReadParameter     : Tc2_MC2.MC_ReadParameter;
17     _fbWriteParameter    : Tc2_MC2.MC_WriteParameter;
18     _fbWriteParameterPersist : Tc2_MC2.MC_WriteParameterPersistent;
    
```

Software Design:

PLC_MOTION_LAYER / GVL_AXIS

```
//-----  
// command and state structure  
//-----  
Ctrl  : ARRAY[1..MAX_AXIS] OF  
        ST_AXIS_CTRL;  
  
State : ARRAY[1..MAX_AXIS] OF  
        ST_AXIS_STATE;  
  
//-----  
// cyclic interface function block  
//-----  
Control : ARRAY[1..MAX_AXIS] OF  
        FB_McAxisCtrl;
```

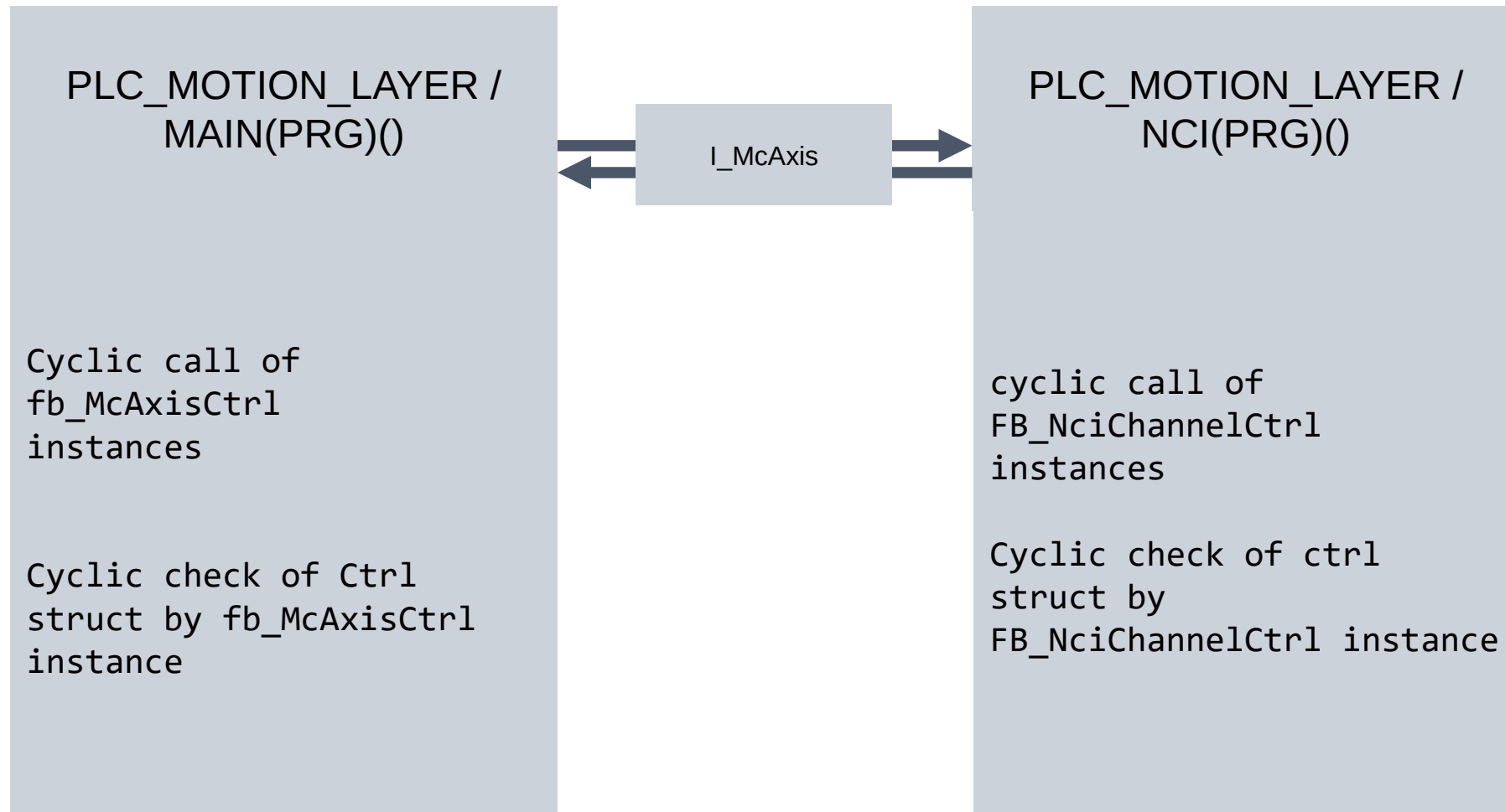
PLC_MOTION_LAYER / GVL_NCI

```
//-----  
// command and state structure  
//-----  
stChannelCtrl  : ARRAY[1..MAX_NCI_CH] OF  
        ST_CTRL_NCI;  
  
stChannelState : ARRAY[1..MAX_NCI_CH] OF  
        ST_STATE_NCI;  
  
//-----  
// cyclic interface function block  
//-----  
fbNciCtrl : ARRAY[1..MAX_NCI_CH] OF  
        FB_NciChannelCtrl;
```

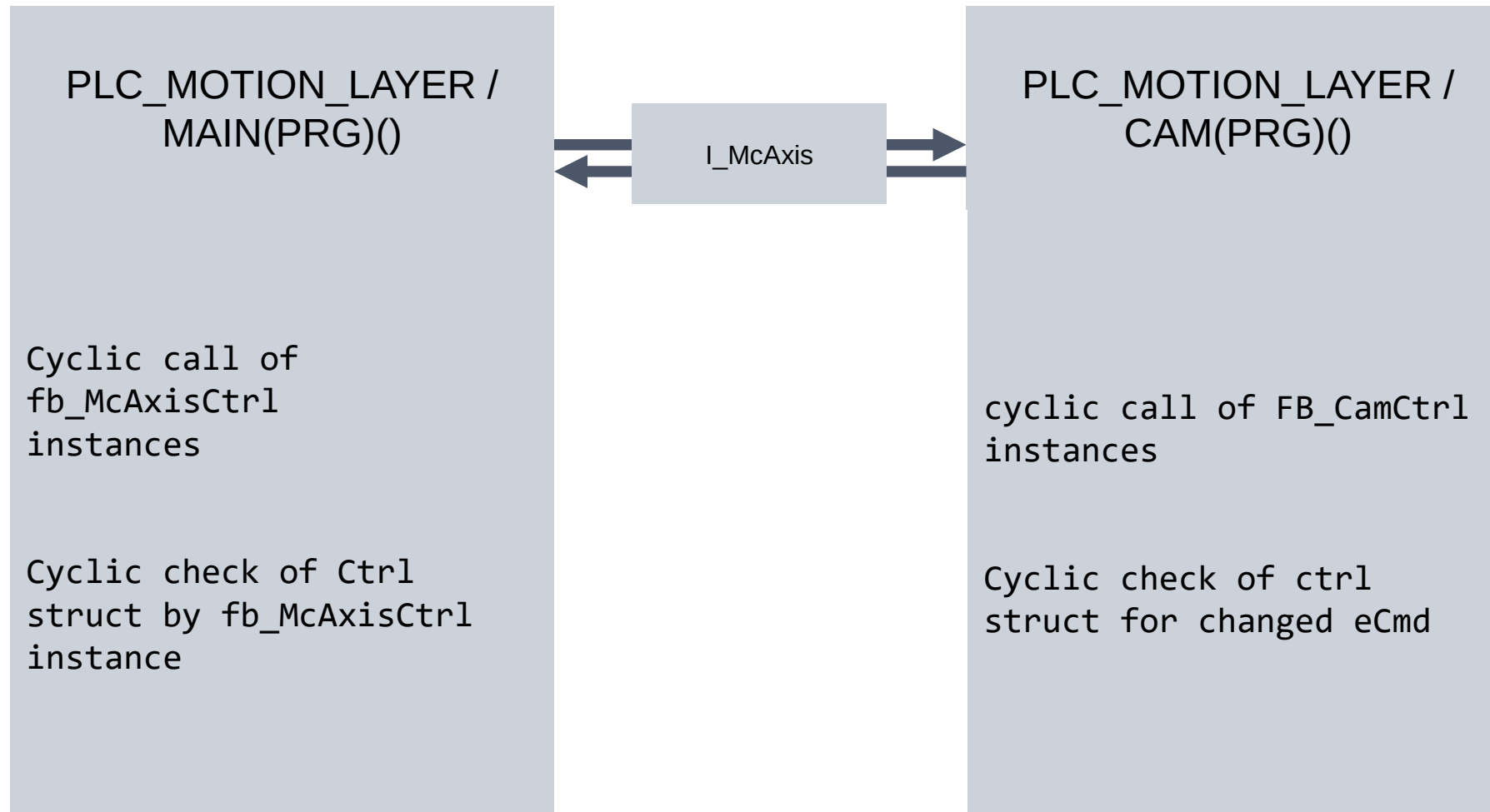
PLC_MOTION_LAYER / GVL_CAM

```
//-----  
// command and state structure  
//-----  
Ctrl  : ARRAY[1..MAX_AXIS] OF  
        ST_CAM_CTRL;  
  
State : ARRAY[1..MAX_AXIS] OF  
        ST_CAM_STATE;  
  
//-----  
// cyclic interface function block  
//-----  
Control : ARRAY[1..MAX_AXIS] OF  
        FB_CamCtrl;
```

Software Design:

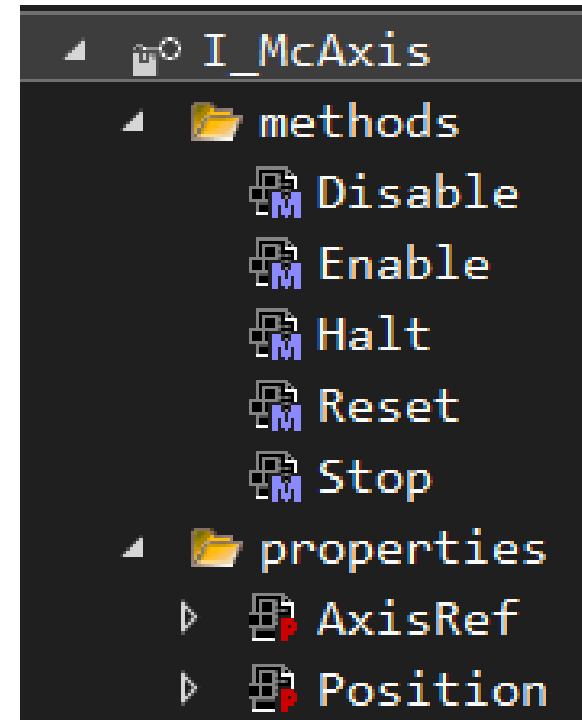


Software Design:



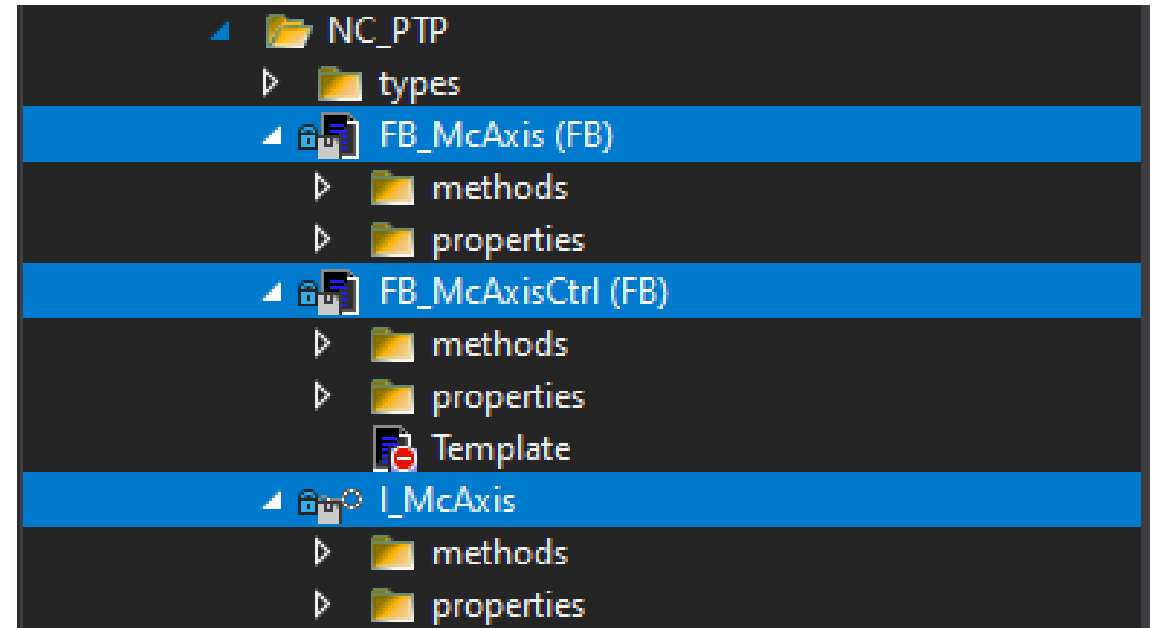
GVL_AXIS:

- I_McAxis: Interface for horizontal access between namespaces
 - Mandatory for using other namespaces
 - Provides essential methods
 - Provides reference to AXIS_REF
 - Direct access to AxisRef.NcToPlc.ActPos



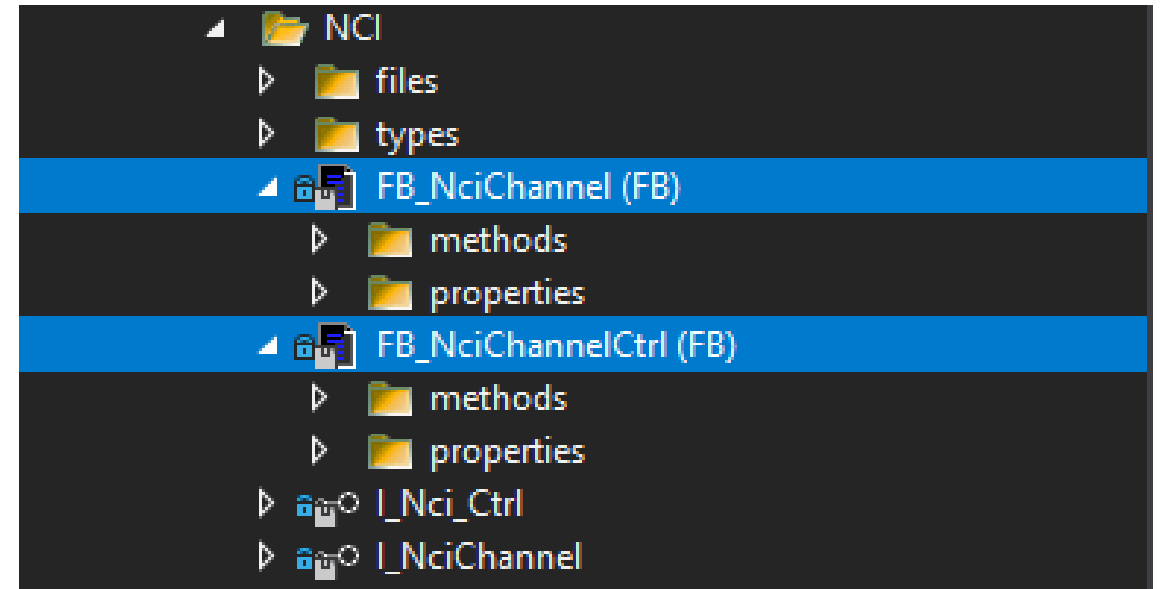
GVL_AXIS:

- McAxis: Point To Point axis
 - Base class **FB_McAxis** wraps Tc2_MC2 function blocks and implements interface
 - **FB_McAxisCtrl** extends base class with cyclic execution wrapper
 - **I_McAxis** is used in advanced motion features (NCI, CAM, XFC)



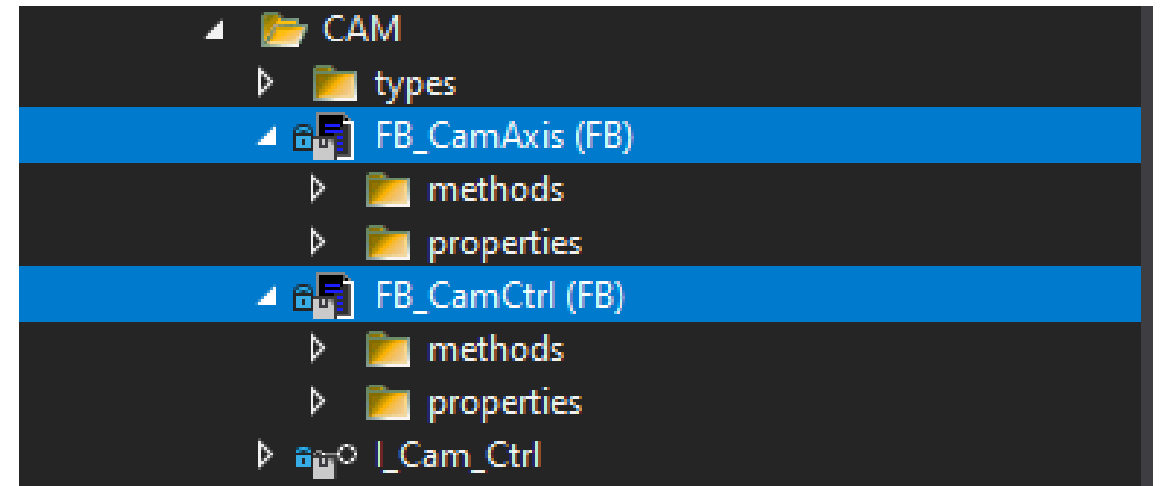
GVL_NCI:

- NCI Channel: XYZ interpolated
 - Base class **FB_NciChannel** wraps Tc2_NCI function blocks and implements interface
- **FB_NciChannelCtrl** extends base class with cyclic execution wrapper and implements interface
- **I_NciChannel** and **I_Nci_Ctrl** are only valid if compiler define is set before compiling project



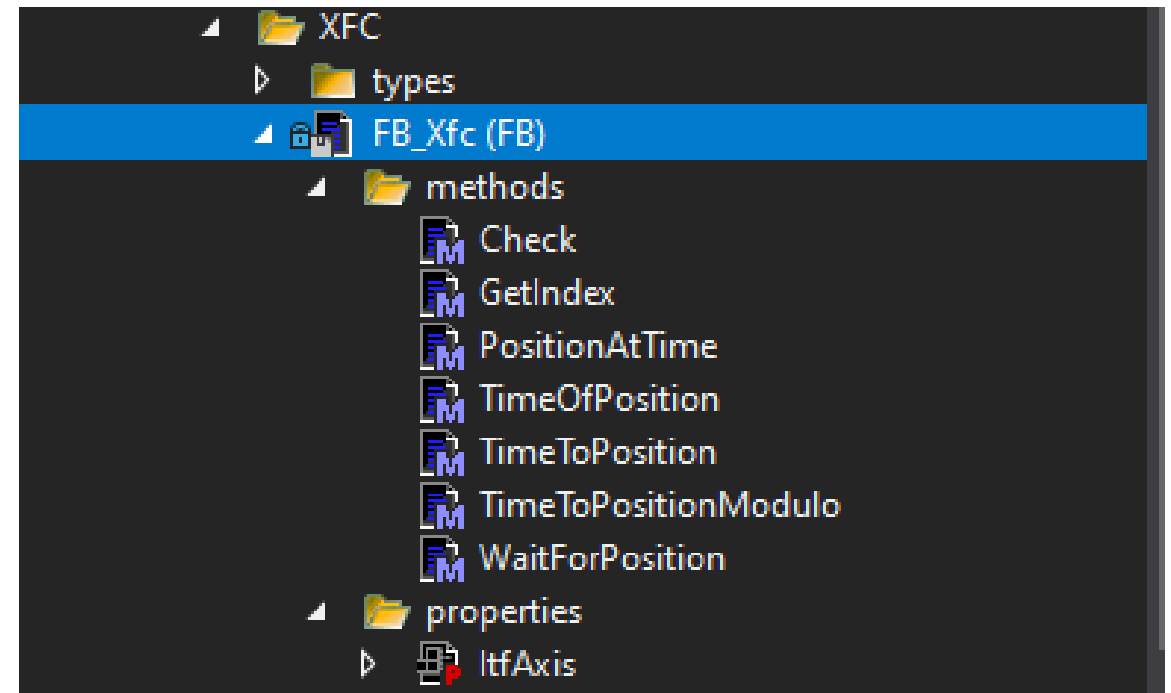
GVL_CAM:

- Camming:
 - Base class **FB_CamAxis** wraps Tc2_MC2_Camming function blocks and implements interface
 - **FB_CamCtrl** extends base class with cyclic execution wrapper and implements interface
 - **I_Cam_Ctrl** is only valid if compiler define is set before compiling project



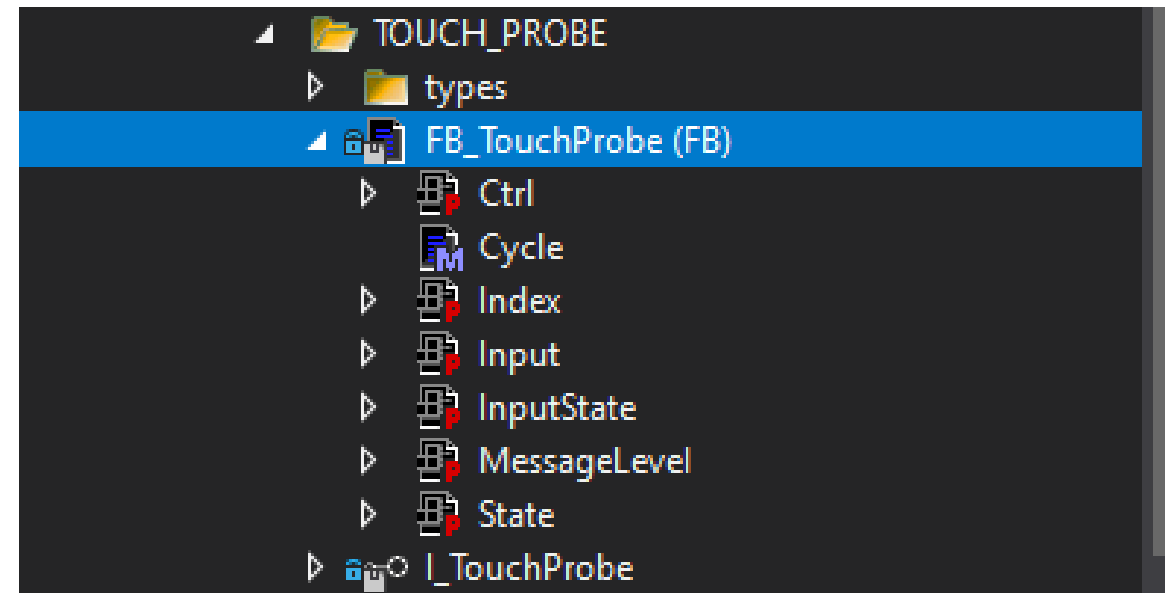
GVL_FUNCTIONS:

- XFC classes:
 - Base class **FB_Xfc** wraps Tc2_MC2_XFC function blocks for Distributed Clock position/time applications



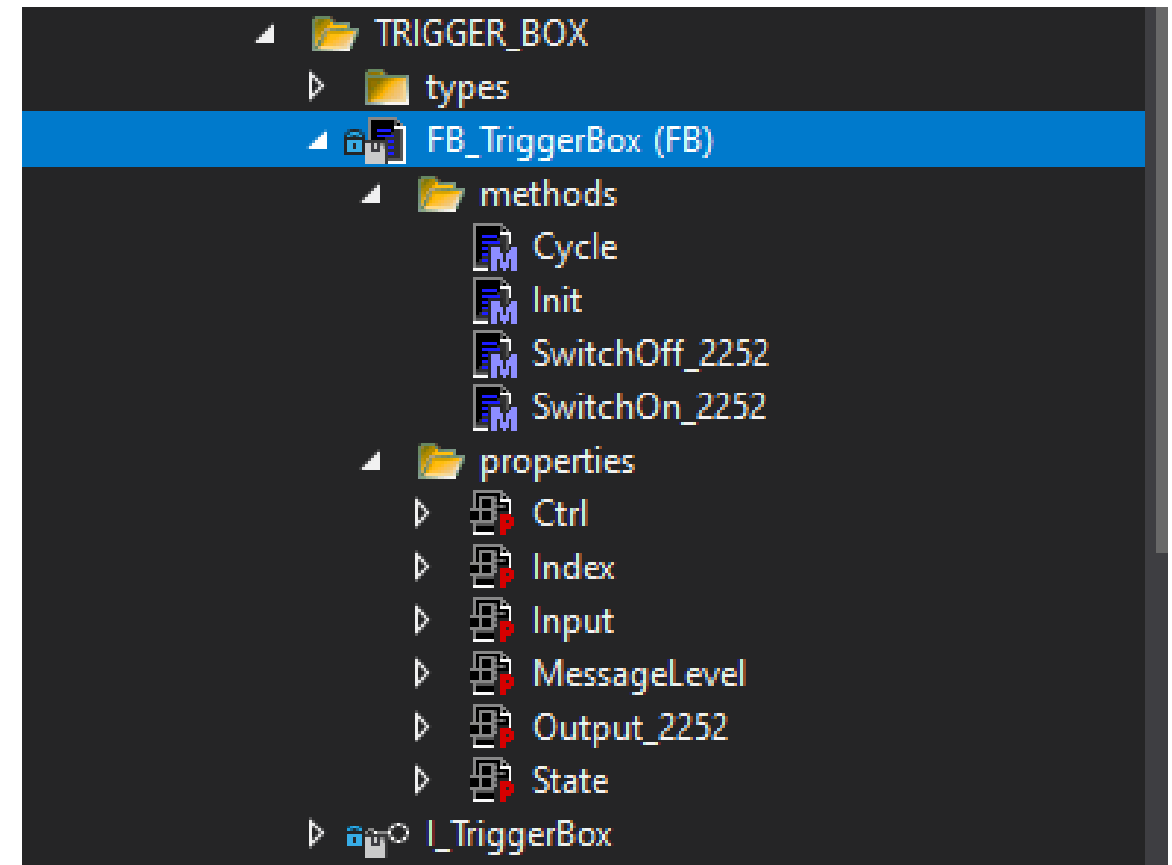
GVL_FUNCTIONS:

- Touch Probe:
 - **FB_TouchProbe** extends **FB_Xfc**
 - Cyclic execution with Ctrl/State pair
 - Must be connected to input device
 - **I_TouchProbe** is only valid if compiler define is set before compiling project



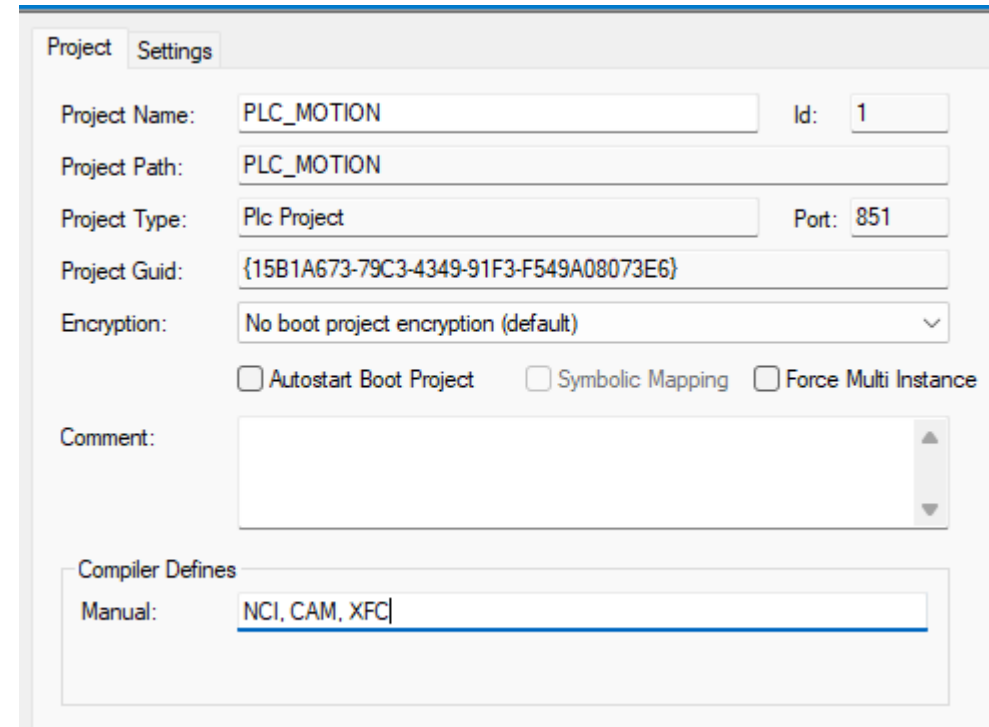
GVL_FUNCTIONS:

- Trigger Box:
 - **FB_TriggerBox** extends **FB_Xfc**
 - Cyclic execution with Ctrl/State pair
 - Must be connected to input/output device
 - **I_TriggerBox** is only valid if compiler define is set before compiling project



Compiler defines / pragmas:

- **BSD**
 - TC-BSD system specific variables are set
- **NCI**
 - NciChannel cyclic interfaces are used
- **CAM**
 - Camming cyclic interfaces are used
- **XFC**
 - TouchProbe and TriggerBox interfaces are used



The screenshot shows the 'Settings' tab of a project configuration window. The 'Project Name' is 'PLC_MOTION' and the 'Id' is '1'. The 'Project Path' is also 'PLC_MOTION'. The 'Project Type' is 'Plc Project' and the 'Port' is '851'. The 'Project Guid' is '{15B1A673-79C3-4349-91F3-F549A08073E6}'. The 'Encryption' is set to 'No boot project encryption (default)'. There are three checkboxes: 'Autostart Boot Project', 'Symbolic Mapping', and 'Force Multi Instance', all of which are currently unchecked. There is a 'Comment' text area which is empty. At the bottom, under 'Compiler Defines', the 'Manual' field contains the text 'NCI, CAM, XFC|'.

Project Name:	PLC_MOTION	Id:	1
Project Path:	PLC_MOTION		
Project Type:	Plc Project	Port:	851
Project Guid:	{15B1A673-79C3-4349-91F3-F549A08073E6}		
Encryption:	No boot project encryption (default)		
☐ Autostart Boot Project ☐ Symbolic Mapping ☐ Force Multi Instance			
Comment:			
Compiler Defines			
Manual:	NCI, CAM, XFC		

Compiler defines / pragmas:

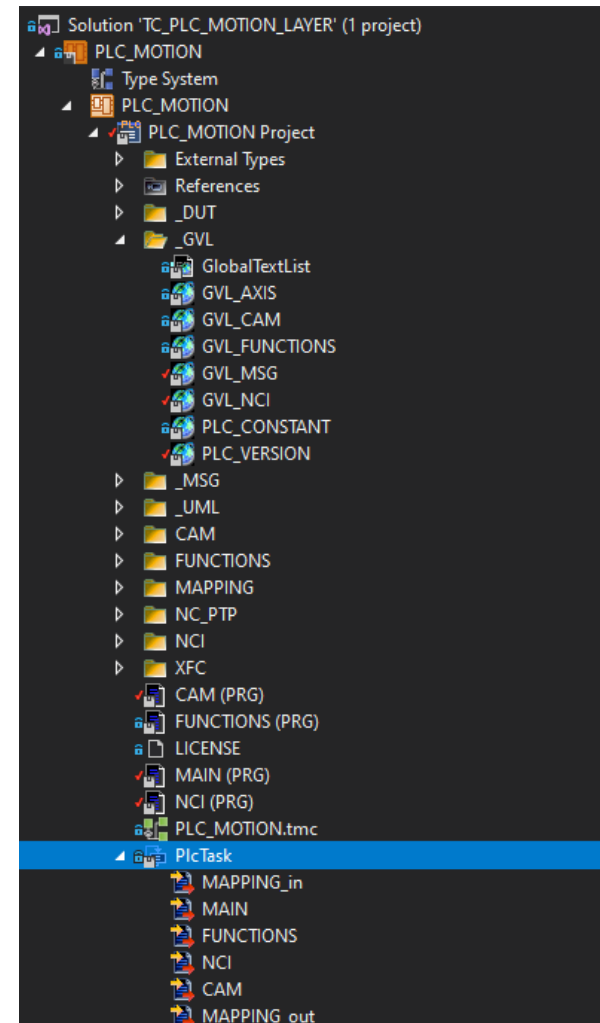
- **AXIS_MAP**
 - MAPPING_in.AxisCtrl is copied onto GVL_AXIS.Ctrl and data structures
 - GVL_AXIS.State is copied onto MAPPING_out.AxisState and info structures

- **CAM_MAP**
 - MAPPING_in.CamControl is copied onto GVL_CAM.Ctrl
 - GVL_CAM.State is copied onto MAPPING_out.CamState and info structures

- **TRIGGER_MAP**
 - TouchProbe and TriggerBox interfaces are used

Cyclic call tree:

- **MAPPING_in(PRG)**
 - Copies mapping data onto input structures
- **MAIN(PRG)**
 - Cyclic call to FB_McAxisCtrl instances
- **FUNCTIONS(PRG)**
 - Cyclic call to TouchProbe and TriggerBox instances
- **NCI(PRG)**
 - Cyclic call to NCI channel instances
- **CAM(PRG)**
 - Cyclic call to FB_CamCtrl instances
- **MAPPING_out**
 - Copies state data onto mapping structures



Logging System:

- Implementation from top down
 - → function based with global timestamp added automatically
- Enumeration based with 4 categories and timestamp
 - → occurrence in strict timestamp order
- Error Id is mirrored directly from called instance
 - → Infosys error numbers can be searched in case of diagnosis
- Optional text for additional information
- Specific logging switch to get more detailed information aside error numbers
- Automated write procedure to ascii formatted file

PLC_MOTION_LAYER project

MIT License

Copyright (c) 2025 HAUD

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.